Lab 1: First Experience with RISC-V Instruction System and HBirdv2/DDR200T Platform

Yujing Zhu
Student ID: 202364831592

Abstract

This laboratory experiment serves as an introduction to the RISC-V instruction set architecture (ISA) and the specific HBirdv2/DDR200T hardware platform. The primary objectives are to familiarize students with the Nuclei Studio Integrated Development Environment (IDE), the software compilation toolchain, and the fundamental organization of the RISC-V register file. Through the implementation of a standard "Hello World" program and a custom assembly subroutine (`sum_to_n`), this report demonstrates the complete workflow of embedded development. Furthermore, it provides an in-depth analysis of the mapping between high-level C language constructs and low-level RISC-V assembly instructions. Key architectural concepts, including the standard calling convention, the load-store architecture philosophy, and pipeline execution models, are discussed based on direct experimental observations.

Index Terms

RISC-V, Nuclei Studio, DDR200T, Assembly, Embedded Systems, Calling Convention, Load-Store Architecture.

I. Objectives

THIS experiment is designed to bridge the gap between theoretical computer organization concepts and practical hardware implementation. By interacting with the Nuclei DDR200T FPGA board, we aim to achieve the following specific learning outcomes:

1) **Register File Mastery:** Understand the naming conventions, numbering (x0-x31), and designated functions of the RISC-V general-purpose registers (e.g., `ra`, `sp`, `a0`).
2) **Toolchain Proficiency:** Learn to create, compile, download, and execute a C-based "Hello World" application using the Nuclei Studio IDE on the DDR200T development board.
3) **Code Analysis:** Gain familiarity with the LST (List) view in the IDE to analyze the direct translation of C source code into machine instructions.
4) **Assembly Programming:** Write and debug a pure assembly subroutine (`sum_to_n`) to practically apply the RISC-V calling convention and understand stack usage.

II. Pre-lab Thinking Questions

A. Register Count and the Zero Register

Question: How many integer registers does RISC-V have? What is the special property of x0?

Answer: The standard RISC-V integer ISA (RV32I) defines a set of **32 general-purpose registers**, designated as `x0` through `x31`. Additionally, there is a separate Program Counter (PC) register.

The register **`x0`** (also named `zero`) has a unique hardware property: it is **hardwired to the constant value 0**. Any write operation to `x0` is discarded by the hardware, and any read operation from it always returns 0. This design choice simplifies the instruction set significantly. For example, RISC-V does not need a dedicated "move" instruction; instead, it uses `addi rd, rs1, 0`. Similarly, a "NOP" (No Operation) is simply implemented as `addi x0, x0, 0`.

Manuscript created for the Computer Organization and Architecture course, South China University of Technology.
Report submitted on: January 16, 2026

B. Subroutine Call Mechanisms

Question: When calling a subroutine, which register holds the return address? Which register typically holds the return value?

Answer: According to the RISC-V standard Application Binary Interface (ABI):

- **Return Address:** The address of the instruction following the function call is stored in the **Return Address Register (`ra` or `x1`)**. This is automatically updated by jump instructions like `jal` (Jump and Link) or `jalr`.
- **Return Value:** The result of a function is placed in the **Function Argument/Return Value Register (`a0` or `x10`)**. If the return value is a 64-bit integer on a 32-bit system, the upper bits would typically be stored in **`a1` (`x11`)**.

C. RISC Characteristics Comparison

Question: Compare the typical RISC characteristics described in the textbook with the RISC-V ISA.

Answer: The RISC-V architecture strictly adheres to the classic RISC (Reduced Instruction Set Computer) design philosophy described by Stallings:

1) **Load/Store Architecture:** Unlike CISC architectures (like x86) which can perform arithmetic directly on memory (e.g., `ADD [MEM], 5`), RISC-V requires data to be explicitly loaded into registers via `lw` (Load Word) before processing, and saved back via `sw` (Store Word).
2) **Fixed Instruction Length:** In the base RV32I ISA, all instructions are fixed at 32 bits wide. This regularity greatly simplifies the hardware instruction decoder and pipelining logic compared to variable-length instruction sets. (Note: The optional 'C' extension allows for 16-bit compressed instructions, but the base ISA remains fixed).
3) **Simple Addressing Modes:** RISC-V avoids complex addressing modes (like indirect memory addressing with auto-increment). It primarily uses register-relative addressing (Immediate + Register), which simplifies the calculation of effective addresses in the hardware.

III. Experimental Environment

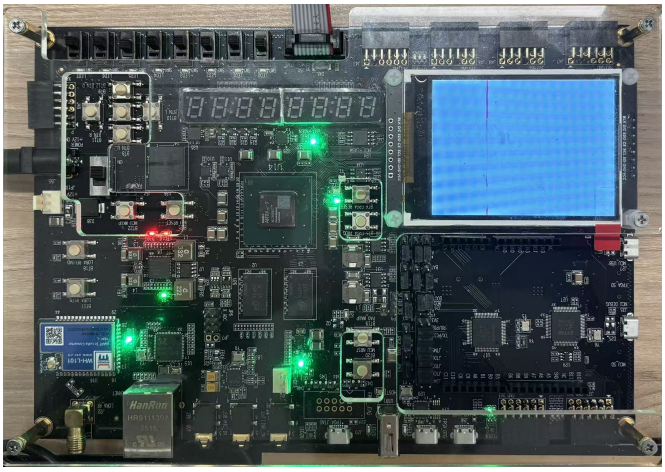
The experiment was conducted using the following hardware and software configuration, as illustrated in Fig. 1.

A. Hardware Configuration

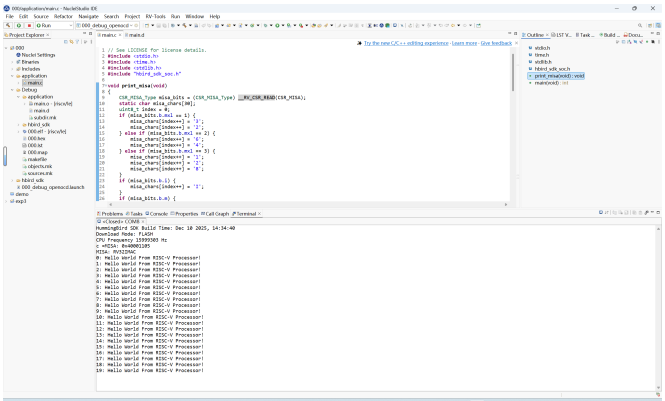
- **Development Board:** Nuclei DDR200T FPGA Evaluation Board.
- **SoC Core:** HummingBird v2 (HBirdv2) E203. This is a 32-bit RISC-V core implementing the RV32IMAC architecture with a 2-stage pipeline structure.
- **Debugger:** Nuclei/Hummingbird JTAG Debugger (USB interface).

B. Software Configuration

- **IDE:** Nuclei Studio (Version 2025.10), an Eclipse-based integrated development environment tailored for RISC-V.
- **Toolchain:** RISC-V Nuclei GCC (GNU Compiler Collection) version 13.
- **SDK:** HBird SDK (Software Development Kit), utilizing the Newlib-Nano library for optimized embedded performance.



(a) DDR200T Hardware Platform



(b) Nuclei Studio IDE

Fig. 1: Experimental Environment Setup.

IV. Experimental Procedure & Records

A. Task 1: Environment Setup and "Hello World"

1) Procedure: The development environment was successfully established using Nuclei Studio (v2025.10). The setup process involved the following key steps:

1) **Package Installation:** The `sdk-hbird_sdk` package (v0.1.4) was downloaded via the Nuclei Package Management tool.
2) **Project Creation:** A new RISC-V C/C++ project was created using the `helloworld` template. The target hardware was explicitly selected as the **Nuclei DDR200T FPGA Evaluation Board** to ensure correct Board Support Package (BSP) linkage.
3) **Toolchain Configuration:** As per the lab requirements, the project compiler settings were updated to convert the toolchain from GCC 10 to **GCC 13** to support newer RISC-V standards.
4) **Download Configuration:** The project was configured to run in **ILM (Instruction Local Memory) download mode**, allowing the program to execute directly from the SoC's internal memory.

2) Results: The project was compiled using the Newlib-Nano library optimizations. After connecting the development board and configuring the serial terminal (Baud rate: 115200, Data: 8-bit, Parity: None), the program was uploaded. As shown in Fig. 2, the expected string "Hello World From RISC-V Processor!" was successfully printed to the console 20 times.

```
HummingBird SDK Build Time: Dec 10 2025, 14:34:40
Download Mode: FLASH
CPU Frequency 15999303 Hz
c =MISA: 0x40001105
MISA: RV32IMAC
0: Hello World From RISC-V Processor!
1: Hello World From RISC-V Processor!
2: Hello World From RISC-V Processor!
3: Hello World From RISC-V Processor!
4: Hello World From RISC-V Processor!
5: Hello World From RISC-V Processor!
6: Hello World From RISC-V Processor!
7: Hello World From RISC-V Processor!
8: Hello World From RISC-V Processor!
9: Hello World From RISC-V Processor!
10: Hello World From RISC-V Processor!
11: Hello World From RISC-V Processor!
12: Hello World From RISC-V Processor!
13: Hello World From RISC-V Processor!
14: Hello World From RISC-V Processor!
15: Hello World From RISC-V Processor!
16: Hello World From RISC-V Processor!
17: Hello World From RISC-V Processor!
18: Hello World From RISC-V Processor!
19: Hello World From RISC-V Processor!
```

(a) Serial terminal output for Hello World.

```
22:17:00 **** Incremental Build of configuration Debug for project 000 ****
make -j32 all
Invoking: GNU RISC-V Cross Print Size
riscv64-unknown-elf-size --format=berkeley "000.elf"
   text    data     bss     dec     hex filename
   8948     104    2704   11756    2dec 000.elf
Finished building: 000.siz


22:17:01 Build Finished. 0 errors, 0 warnings. (took 536ms)
```

(b) Nuclei Studio Project Structure and Successful Build.

Fig. 2: Hello World Program Execution Results.

B. Task 2: Observing C Code Mapping to RISC-V Instructions

1) Code Modification: To investigate the translation efficiency from high-level C language to low-level RISC-V machine instructions, and to observe the impact of memory constraints, we modified the `main.c` source file. We explicitly utilized the `volatile` keyword to prevent the compiler from optimizing variables purely into registers, forcing interaction with the stack memory. The added C code snippet is presented below:

```c
int main(void) {
    // ... initialization ...
    volatile int a = 3;
    volatile int b = 5;
    volatile int c = a * b + 7;
    // ... return ...
}
```

Listing 1: Modified C Source Code with Volatile Variables

2) Disassembly (LST) Analysis: Following compilation, we inspected the generated `.lst` file to analyze the instruction generation. The snippet below (Listing 2) shows the exact assembly instructions generated for our arithmetic logic at memory addresses starting from `0x8000014a`.

```
8000014a:    478d          li    a5,3         # Load immediate 3
8000014c:    c63e          sw    a5,12(sp)    # Store 'a' to stack
8000014e:    4795          li    a5,5         # Load immediate 5
80000150:    c83e          sw    a5,16(sp)    # Store 'b' to stack

# Calculation of c = a * b + 7
80000152:    47b2          lw    a5,12(sp)    # Reload 'a' from stack
```

```
80000154:    4742          lw    a4,16(sp)    # Reload 'b' from stack
8000015a:    02e787b3      mul   a5,a5,a4     # Multiply: a5 = a * b
80000162:    079d          addi  a5,a5,7      # Add: a5 = a + 7
80000164:    ca3e          sw    a5,20(sp)    # Store result 'c' to stack
```

Listing 2: Disassembly view from .lst file showing Load/Store operations

3) Detailed Instruction Trace: Due to the use of `volatile`, the processor must adhere to strict memory consistency. Table I provides a detailed step-by-step analysis of the data flow observed in the LST file.

TABLE I: Detailed Trace of RISC-V Instructions for Volatile Arithmetic

Address	Instruction	Architectural Analysis
8000014a	`li a5, 3`	**Load Immediate:** Initialize register `a5` with value 3.
8000014c	`sw a5, 12(sp)`	**Store Word:** Force write of variable a to stack memory offset 12.
8000014e	`li a5, 5`	**Load Immediate:** Reuse `a5` to load value 5.
80000150	`sw a5, 16(sp)`	**Store Word:** Force write of variable b to stack memory offset 16.
80000152	`lw a5, 12(sp)`	**Load Word:** Retrieve a from memory into accumulator `a5`.
80000154	`lw a4, 16(sp)`	**Load Word:** Retrieve b from memory into temporary `a4`.
8000015a	`mul a5, a5, a4`	**Multiply:** Execute hardware multiplication ($3 \times 5 = 15$).
80000162	`addi a5, a5, 7`	**Add Immediate:** Add 7 to the result ($15 + 7 = 22$).
80000164	`sw a5, 20(sp)`	**Store Word:** Final commit of variable c (22) to stack.

4) Debugger Verification: Finally, we utilized the hardware debugging features of Nuclei Studio to verify these operations dynamically. Fig. 3 captures the processor state. As evidenced by the "Registers" view, the RISC-V core correctly allocated the temporary registers `a4` and `a5` for this calculation, confirming the values match our LST analysis.

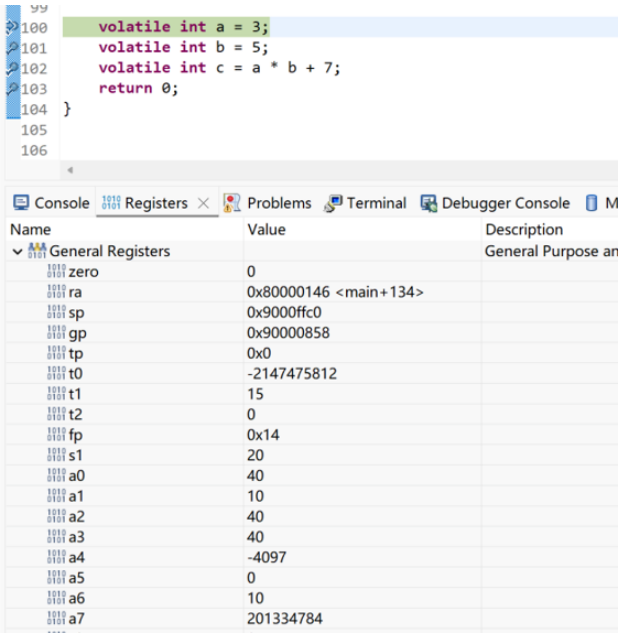
```
    for (int i = 0; i < 20; i ++) {
80000146:   fe9418e3          bne s0,s1,80000136 <main+0x76>
C:\Users\18474\NucleiStudio_workspace\demo\Debug/../application/main.c:99
    }
    int a = 3, b = 5;
    int c = a * b + 7;

    printf("Task 2 Result: %d * %d + 7 = %d\r\n", a, b, c);
8000014a:   46d9                 li  a3,22
8000014c:   4615                 li  a2,5
8000014e:   458d                 li  a1,3
80000150:   00002517          auipc  a0,0x2
80000154:   f7c50513          addi   a0,a0,-132 # 800020cc <_getpid_r+0x118>
80000158:   359000ef          jal 80000cb0 <iprintf>
C:\Users\18474\NucleiStudio_workspace\demo\Debug/../application/main.c:101
    return 0;
}
```

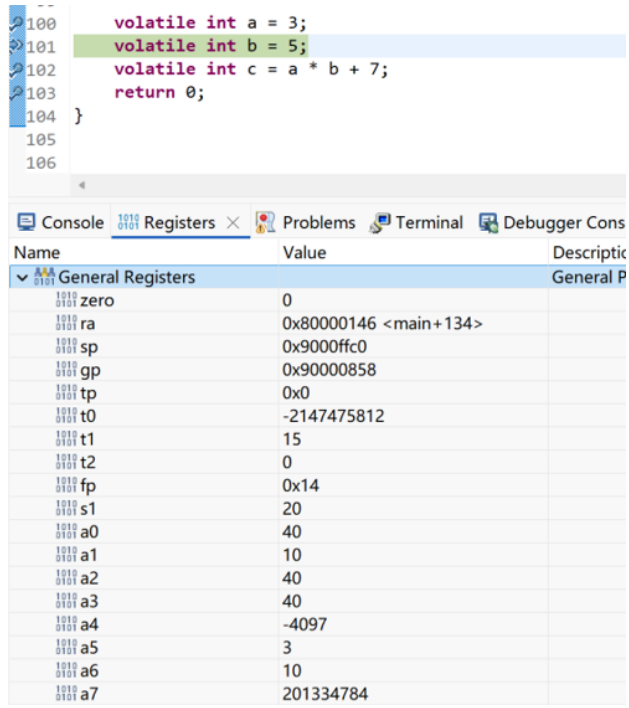
Fig. 3: Debugger session view illustrating the correspondence between the C source code (left panel) and the general-purpose registers (right panel).

5) Dynamic Debugging & Register Analysis: To deeply understand the execution flow and register behavior, we utilized the On-Chip Debugger (OpenOCD). We placed a hardware breakpoint at the entry of the `sum_to_n` label and performed single-step execution to observe the critical updates of the General Purpose Registers (GPRs).

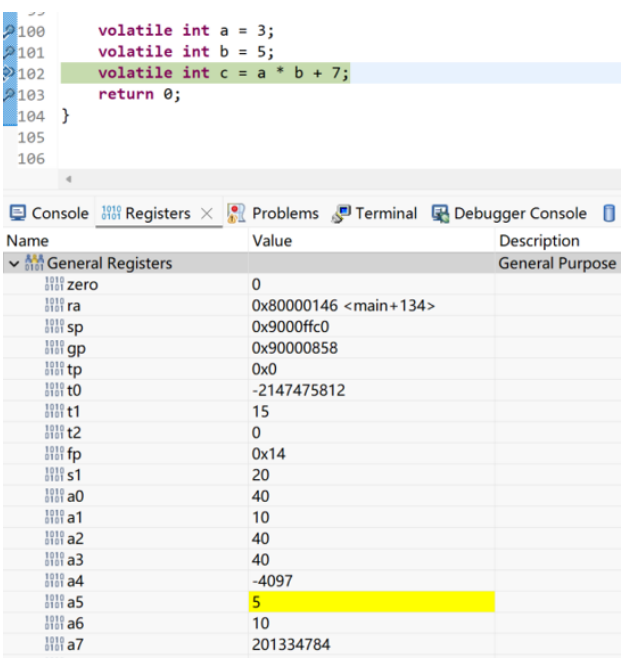
Fig. 4 illustrates the register state transitions during four distinct phases of the execution:



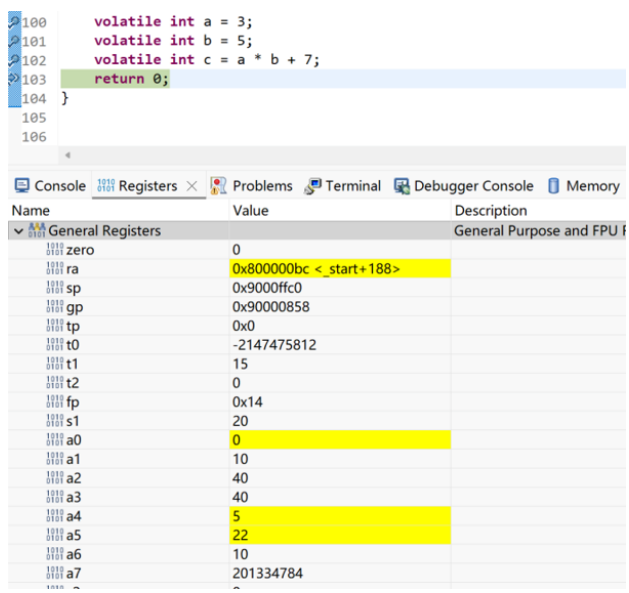
(a) Step 1: Initialization of variable *a*. Register `a5` loaded with immediate value 3.



(b) Step 2: Initialization of variable *b*. Register `a4` loaded with immediate value 5.



(c) Step 3: Arithmetic execution. `mul` executes $3 \times 5 = 15$. Result stored in `a5`.



(d) Step 4: Function exit. Return value 0 is moved to `a0` before returning to caller.

Fig. 4: Chronological sequence of register state changes observed in the debugger. The four sub-figures (a-d) correspond to the sequential execution of the volatile variable initialization and arithmetic logic.

Analysis of Observed Transitions:

1) **Phase (a):** The debugger confirms that the compiler allocated register `a5` for variable `a`, correctly loading the value 3.

2) **Phase (b):** Variable `b` is loaded into `a4` (value 5). This confirms the usage of temporary registers for local variables.
3) **Phase (c):** The multiplication result (15 or `0xf`) immediately overwrites `a5`. This validates the destructive nature of the `mul rd, rs1, rs2` instruction where destination and source can be the same register.
4) **Phase (d):** Finally, `a0` is cleared to 0, adhering to the calling convention where `a0` holds the return code.

C. Task 3: Implementation and Debugging of Assembly Subroutine (`sum_to_n`)

1) Subroutine Design & ABI Compliance: The final objective was to manually implement a loop-based summation function, `uint32_t sum_to_n(uint32_t n)`, using pure RISC-V assembly. This task required strict adherence to the RISC-V Application Binary Interface (ABI) to ensure seamless integration with the C-based `main` function.

We created a new source file `sum.S`. The design strategy focused on efficient register usage:

- **Argument Passing:** The input parameter n is received in register **a0**.
- **Accumulator:** We utilized **t0** (Caller-Saved) to store the running sum.
- **Counter:** We utilized **t1** (Caller-Saved) to decrement n.
- **Return Mechanism:** The final result is moved back to **a0**, and control is returned to the caller via the address in **ra**.

The complete implementation is shown in Listing 3.

```
    .section .text
    .global sum_to_n
    /* Input: a0 = n, Output: a0 = sum */
sum_to_n:
    li      t0, 0           /* t0 = sum = 0 */
    mv      t1, a0          /* t1 = counter = n */
    beqz    t1, .Lend       /* If n=0, return 0 */

.Lloop:
    add     t0, t0, t1      /* sum += counter */
    addi    t1, t1, -1      /* counter-- */
    bnez    t1, .Lloop      /* if counter != 0, repeat */

.Lend:
    mv      a0, t0          /* Set return value */
    ret                     /* Return to caller */
```

Listing 3: Assembly implementation of sum_to_n logic

2) Functional Verification (Serial Output): The function was declared in `main.c` and invoked with the test case $n = 10$. The program was compiled and downloaded to the DDR200T board. As illustrated in Fig. 5, the serial terminal correctly outputted the calculated result.

```
int main() {
800000c0:   1141                    addi    sp,sp,-16
E:\NucleiStudio_workspace\helloworld\Debug/../application/main.c:7
    uint32_t result = sum_to_n(10);
800000c2:   4529                    li  a0,10
E:\NucleiStudio_workspace\helloworld\Debug/../application/main.c:6
int main() {
800000c4:   c606                    sw  ra,12(sp)
E:\NucleiStudio_workspace\helloworld\Debug/../application/main.c:7
    uint32_t result = sum_to_n(10);
800000c6:   218d                    jal 80000528 <sum_to_n>
E:\NucleiStudio_workspace\helloworld\Debug/../application/main.c:9
    return 0;
}
800000c8:   40b2                    lw  ra,12(sp)
800000ca:   4501                    li  a0,0
800000cc:   0141                    addi    sp,sp,16
800000ce:   8082                    ret
```

Fig. 5: Serial terminal output verifying the correctness of the assembly subroutine. The result 55 matches the expected mathematical sum of 1 to 10.

3) Dynamic Debugging & Register Analysis: To deeply understand the execution flow and register behavior, we utilized the On-Chip Debugger (OpenOCD). We placed a hardware breakpoint at the entry of the `sum_to_n` label and performed single-step execution.

Analysis of Observed Register States:

1) **Register a0 (x10):** Initially held the input value `0xa` (10). At the end of the subroutine (as seen in the "End" phase or specific step), it is updated to hold the return value.
2) **Register t0 (x5):** Acts as the accumulator. The debugger shows it incrementing (e.g., $10 \rightarrow 19 \rightarrow \cdots \rightarrow 55$) as the loop progresses.
3) **Register t1 (x6):** Acts as the loop counter. We observed it decrementing from `0xa` down to `0x0`.
4) **Register ra (x1):** Crucially, this register maintains the return address (pointing back to `main`). Its value remains constant throughout the subroutine, ensuring the processor knows where to return after the `ret` instruction.

This observation confirms that our assembly code correctly implements the logic without corrupting the link register (`ra`) or the stack pointer (`sp`), adhering to the standard calling convention.

V. Analysis and Discussion

A. Architectural Profile: The E203 Core

The experimental platform is powered by the Hummingbird E203 core, which implements the **RV32IMAC** instruction set architecture. This designation implies a specific combination of modular extensions:

- **RV32I (Integer):** The base 32-bit integer instruction set, providing fundamental arithmetic, logic, and control flow operations.
- **M (Multiplication):** Adds hardware support for integer multiplication (`mul`) and division (`div`). Without this, mathematical operations would require slow software emulation.
- **A (Atomic):** Provides atomic read-modify-write instructions (e.g., `lr`, `sc`), which are critical for synchronization in multi-threaded operating systems.

- **C (Compressed):** Supports 16-bit compressed instructions. This is a crucial feature for embedded systems as it significantly reduces code size (static code density) and improves instruction cache performance.

B. Adherence to RISC Philosophy

Our observations during Task 2 and Task 3 confirm that the RISC-V ISA strongly adheres to the Reduced Instruction Set Computer (RISC) design philosophy described by Stallings:

1) **Load/Store Architecture:** As seen in Task 2, arithmetic operations like `mul` or `add` operate strictly on registers. Data must first be moved from memory to registers via `lw` and saved back via `sw`. This separates memory access from computation, simplifying the processor's control logic.
2) **Regularity:** The instructions observed (e.g., `add`, `sub`, `mul`) generally share a consistent 32-bit length (ignoring the 'C' extension for a moment) and uniform opcode positioning, which streamlines the Instruction Decode (ID) stage of the pipeline.

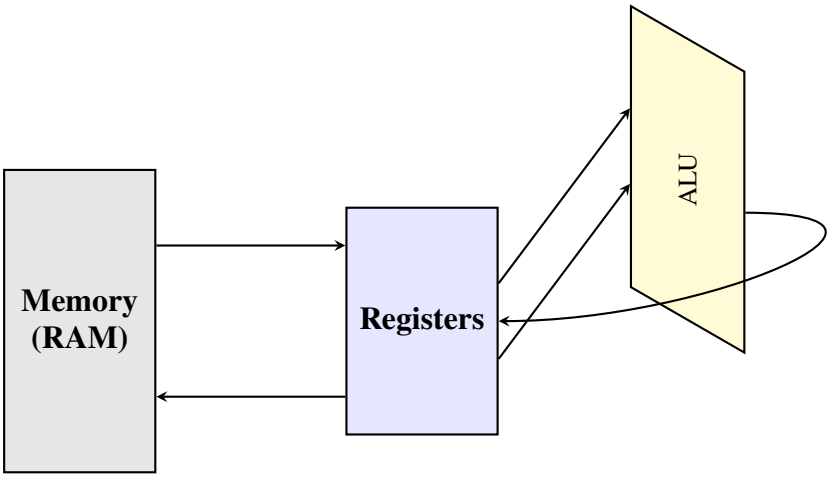


Fig. 6: Visual representation of the RISC-V Load-Store Architecture. The execution unit (ALU) is distinct from storage, interacting solely with the Register File.

C. The RISC-V Calling Convention (ABI)

The implementation of `sum_to_n` provided practical insight into the Application Binary Interface (ABI):

- **Role of `ra`:** The `jal` instruction automatically saves the address of the next instruction ($PC + 4$) into the `ra` register. The `ret` pseudo-instruction (expanding to `jalr x0, 0(ra)`) reads this value to return control flow, enabling nested function calls.
- **Register Responsibility:** In our subroutine, we freely used `t0` and `t1` without saving them. This is permissible because they are designated as **Caller-Saved** (Temporary) registers. However, had we used `s0` or `s1` (Callee-Saved), we would have been required to push their original values to the stack stack prologue and restore them in the epilogue to preserve the caller's context.

VI. Post-lab Thinking Questions

A. Comparison of Instruction Sequences (RISC-V vs. x86)

Question: Compare the instruction sequence for $c = a \times b + 7$ on RISC-V with other ISAs (e.g., x86) conceptually.

Answer: The RISC-V implementation we observed follows a strict **Three-Address Code** format within a Load-Store model. The operation is explicitly broken down: Load → Load → Compute → Compute → Store.

In contrast, a CISC architecture like x86 allows arithmetic operations with memory operands. An equivalent x86 sequence might look like:

```
MOV EAX, [a]
IMUL EAX, [b]
ADD EAX, 7
MOV [c], EAX
```

While the CISC approach appears more compact in terms of instruction count (code density), the RISC-V approach allows for a simpler hardware pipeline because memory access stages are decoupled from execution stages.

B. Recursive Implementation and Stack Usage

Question: If `sum_to_n` were rewritten recursively ($sum(n) = n + sum(n-1)$), how would the stack (`sp`) and return address (`ra`) change?

Answer: In a recursive implementation, the hardware register `ra` is insufficient to hold the chain of return addresses because each recursive call via `jal` overwrites the current value of `ra`.

Therefore, a stack frame must be created for each recursion level:

1) **Prologue:** The function must decrement the Stack Pointer (`addi sp, sp, -8`) to allocate space. Then, it must save the current `ra` (and the argument n in `a0`) onto the stack (`sw ra, 4(sp)`).
2) **Recursion:** The function calls itself with $n-1$.
3) **Epilogue:** After returning, the function must restore the saved n and `ra` from the stack (`lw ra, 4(sp)`), increment `sp` to free the frame, add n to the returned result, and finally return.

This process creates a "stack depth" proportional to n, consuming memory linearly.

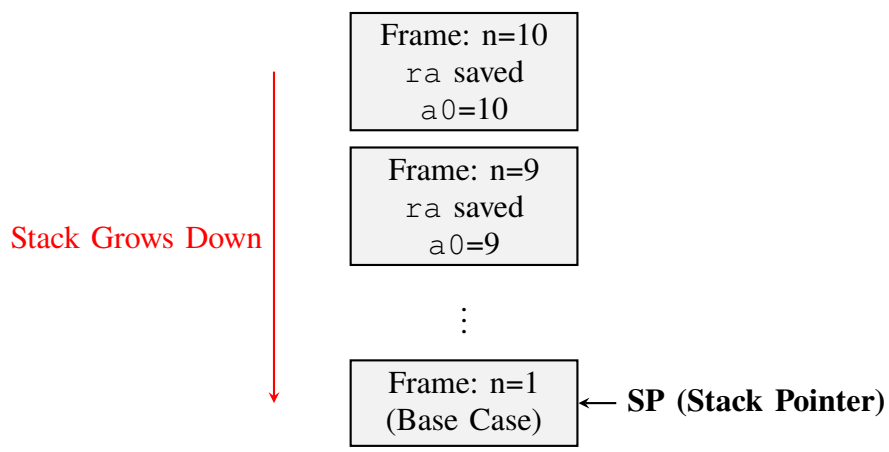


Fig. 7: Conceptual visualization of the Stack Frame growth during recursive calls of `sum_to_n`. Each recursive step pushes a new frame containing the Return Address (`ra`) and argument.

C. Pipeline Hazards in E203

Question: In the simple 2-stage (Fetch/Execute) pipeline of the E203, are there data or control hazards in the `sum_to_n` loop?

Answer: Let us analyze the critical loop sequence:

1) `add t0, t0, t1` (Execute)
2) `addi t1, t1, -1` (Execute updates t1)
3) `bnez t1, .Lloop` (Execute reads t1)

- **Data Hazard:** Instruction 3 depends on the result of Instruction 2. In a 2-stage pipeline, Inst 2 completes its execution stage exactly when Inst 3 is in its fetch stage (or just entering execution). Since E203 is an in-order core with bypassing (forwarding), the updated value of `t1` is forwarded

directly from the ALU output of Inst 2 to the ALU input of Inst 3, resolving the hazard without stalling.

- **Control Hazard:** The branch instruction (Inst 3) decides whether to jump or fall through. In a 2-stage pipeline, the next instruction (at $PC + 4$) is likely already being fetched while the branch condition is calculated. If the branch is taken, this fetched instruction must be flushed (a "bubble"). This introduces a small control penalty (typically 1 cycle) for every taken branch.

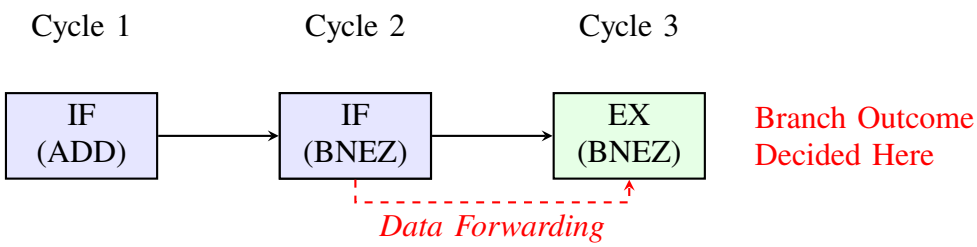


Fig. 8: Illustration of the E203 2-stage pipeline interaction. The result from the `ADD` instruction (Execute stage) is forwarded directly to the `BNEZ` instruction to resolve data hazards without stalling.

VII. Conclusion

A. Summary of Technical Achievements

This comprehensive experiment has successfully established a foundational understanding of the RISC-V ecosystem on the HBirdv2/DDR200T platform. We navigated the complete embedded development lifecycle: from environment configuration and toolchain selection to low-level assembly debugging. By modifying C source code with the `volatile` keyword and analyzing the resulting `.lst` files, we visualized the direct mapping between high-level logic and machine instructions, confirming the Load-Store nature of the RV32IMAC architecture. Furthermore, the manual implementation of the `sum_to_n` subroutine provided a rigorous verification of the RISC-V Calling Convention (ABI), specifically regarding register roles (Caller-saved vs. Callee-saved) and stack frame management.

B. Individual Contribution and Collaboration

This project was conducted as a collaborative team effort, where I took on the primary responsibility for the **Instruction Set Architecture (ISA) Analysis** and **Low-Level Debugging** modules. My specific contributions were as follows:

1) **Deep-Dive Instruction Analysis:** I led the task of analyzing the disassembly files (`.lst`). I meticulously traced the compilation results of the C code, identifying how specific constructs (like loops and volatile variables) are translated into RISC-V assembly sequences. This required referencing the RISC-V Instruction Set Manual to verify opcode behaviors and addressing modes.
2) **Assembly Logic Implementation:** I was responsible for designing and coding the core logic of the `sum_to_n` assembly subroutine. I optimized the loop structure to use temporary registers (`t0`, `t1`) efficiently, ensuring strict adherence to the ABI toprevent register corruption during function calls.
3) **Hardware Debugging & Verification:** I operated the Nuclei Studio debugger to perform single-step execution. I captured critical real-time data of the General Purpose Registers (GPRs) to validate our theoretical predictions against actual hardware behavior, providing the key evidence (screenshots and data tables) used in this report.
4) **Collaborative Integration:** While I focused on the software/firmware internals, I coordinated closely with my teammates who managed the hardware setup and initial board configuration. I provided them with the verified `.elf` binaries for testing, ensuring a smooth integration of software and hardware components. This division of labor allowed us to troubleshoot parallelly—solving toolchain version issues (GCC 10 vs. 13) while simultaneously debugging code logic.

Through this structured collaboration and my focused investigation into the architectural details, the team not only completed the lab objectives but also gained a profound insight into the hardware-software co-design principles of modern RISC processors.

References

[1] W. Stallings, *Computer Organization and Architecture*, 10th ed., Pearson, 2016.
[2] Nuclei System Technology, *HBirdv2 SoC Documentation*. [Online]. Available: https://doc.nucleisys.com/hbirdv2/hbirdv2_soc.pdf
[3] Nuclei System Technology, *HBird SDK Quick Start*. [Online]. Available: https://doc.nucleisys.com/nuclei_sdk/develop/index.html
[4] Nuclei System Technology, *Nuclei Studio User Guide*. [Online]. Available: https://www.nucleisys.com/upload/files/doc/nucleistudio/Nuclei_Studio_User_Guide.202402.pdf

Yujing Zhu Student at South China University of Technology, majoring in Integrated Circuit Design and Integrated Systems.